

Description

Editorial

Concepts

Select a Solution Concept to Visualize:

Sliding Window
(Optimal)

$O(N)$ | $O(K)$, where K is the number of unique characters

This approach involves maintaining a sliding window of unique characters and updating the length accordingly.

Input Configuration

Import from LeetCode URL

<https://leetcode.com/problems/longest-substring-without-repeating-characters/>

Fetch

Test Case Input (array, grid, or edges)

abcabcbb

► Advanced Batch Test Cases

Auto-Instrument

Generate Trace & AI

Commentary

Code Editor

C++17

Skip AI

```
1 class Solution {
2 public:
3     int lengthOfLongestSubstring(string s) {
4         int n = s.length();
5         unordered_map<char, int> charIndex;
6         int left = 0, maxLength = 0;
7         for (int right = 0; right < n; right++) {
8             focus_pointer("left", left);
9             focus_pointer("right", right);
10            visit(right);
11            if (charIndex.count(s[right])) {
12                compare(left, charIndex[s[right]]);
13                // Shift left pointer past the last seen
14                // index of the duplicate
15                left = max(left, charIndex[s[right]] +
16                        1);
17            }
18            charIndex[s[right]] = right;
19            maxLength = max(maxLength, right - left +
20                        1);
21            resolve(right, maxLength);
22        }
23        return maxLength;
24    }
25 }
```



Visualizer Animation

left

right

a	b	c	a	b	c	b	b
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]

Algorithm finished.

Step



←
Prev

Step: 46
/ 46

Next
→

▶ Auto-play

Speed:

Normal (1s)



AI Complexity Analysis

TIME COMPLEXITY

$O(N)$

SPACE COMPLEXITY

$O(\min(N, M))$

Bottlenecks & Dynamic Allocations

The primary bottleneck is the use of `unordered_map<char, int>` which can lead to slow lookups if there are many unique characters. This could be improved by using a more efficient data structure.

💡 AI Judge & Optimization Steering

While this solution has good time complexity, it uses extra space for the hashmap. If memory is a concern, consider using a single-pointer approach that keeps track of the last seen character's index.

Interactive Growth Curve



